# La Union hace la Fuerza


## Contents

# 1 Data Structures

## 1.1 Priority Queue de minimos

```
priority_queue<int, vector<int>, greater<int>> pq;
pq.push(x);// Inserta el elemento x
pq.top();// Devuelve el menor elemento (minimo)
pq.pop();// Elimina el menor elemento
pq.empty();// Retorna true si esta vacia
pq.size();// Numero de elementos
```

## 1.2 Priority Queue de maximos

```
priority_queue<int> pq;
pq.push(x);// Inserta el elemento x
pq.top();// Devuelve el mayor elemento (maximo)
pq.pop();// Elimina el mayor elemento
pq.empty();// Retorna true si esta vacia
pq.size();// Numero de elementos
//Coordenadas cilindricas entra fija
```

## 1.3 Tabla Aditiva - Partial Sum o Prefix Sum

```
vector<int> v = {1, 2, 3, 4, 5};
vector<int> prefix(v.size());
partial_sum(v.begin(), v.end(), prefix.begin()//posicion desde la que se
//construira);
for (int x : prefix)
    cout << x << "␣";
// Output: 1 3 6 10 15
```

# 2 Operaciones con bits en C++

## 2.1 Funciones utilices de C++

### 2.1.1 popcount

Para obtener el numero de bits encendidos en un entero

```
 cout << __builtin_popcount(n) << '\n'; // siendo n un int
```

Version para numeros de 64bits (long long)

```
cout << __builtin_popcountll(n) << '\n'; //  siendo n un long long
```

Hack bits

```
/*
 * Bit Hacks
 * Useful techniques for working with bit masks.
 * Widely used in competitive programming for set problems, DP with
     bitmasks, combinatorics, etc.
 */

#include <bits/stdc++.h>
using namespace std;

using ull = unsigned long long;

/*
 * next_bits_permutation(x)
 * Returns the next mask with the same number of set bits (1s) in
     lexicographical order.
 * Useful for generating all combinations of k elements as bit masks.
 *
 * Example:
 * x = 0b00111 (represents a combination of 3 elements)
 * next_bits_permutation(x) -> 0b01011, then 0b01101, etc.
 */
ull next_bits_permutation(ull x) {
    ull c = __builtin_ctzll(x);         // Count trailing zeros in x
    ull r = x + (1ULL << c);            // Add 1 to the least
        significant set bit
    return (r ^ x) >> (c + 2) | r;      // Calculate the next
        permutation
}

/*
 * subsets(s)
 * Iterates over all proper subsets (does not include s) of a given set
     s.
 * The set s is represented as a bit mask.
 *
 * Example:
 * s = 0b1011 (set {0,1,3})
 * Will iterate over subsets: 1010, 1001, 1000, 0011, 0010, 0001
 */
void subsets(ull s) {
    for (ull x = s; x;) {
        --x &= s;
```

```
        // You can work with each subset x here
        // For example, print it:
        // cout << bitset<4>(x) << '\n';
    }
}
```

### 2.2 Gcd Bits

```
//gcd using binary algorithm
int binary_gcd(int a, int b) {
    if (a == 0) return b;
    if (b == 0) return a;

    int shift;
    for (shift = 0; ((a | b) & 1) == 0; ++shift) {
        a >>= 1;
        b >>= 1;
    }

    while ((a & 1) == 0)
        a >>= 1;

    do {
        while ((b & 1) == 0)
            b >>= 1;

        if (a > b) swap(a, b);
        b = b - a;
    } while (b != 0);

    return a << shift;
}
```

## 3 Grafos

### 3.1 Revision 8 adyacentes

```
vector <int> dx{-1,1,-1,1,0,0,1,-1};
vector <int> dy{-1,1,0,0,1,-1,-1,1};
```

### 3.2 Algoritmo Dijkstra

```
/*
 * Dijkstra's Algorithm
 * Finds the minimum distances from a source node in a weighted graph
     with no negative weights.
 * Complexity: O((V + E) log V)
 */

#include <bits/stdc++.h>
using namespace std;

const int INF = 1e9;
using pii = pair<int, int>; // {dist, node}

vector<int> dijkstra(int n, int src, const vector<vector<pii>>& adj) {
    vector<int> dist(n, INF);
    priority_queue<pii, vector<pii>, greater<>> pq;
    dist[src] = 0;
    pq.emplace(0, src);

    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;
        for (auto [v, w] : adj[u]) {
            if (dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                pq.emplace(dist[v], v);
            }
        }
    }
    return dist;
}
```

### 3.3 Algoritmo Bellman-Ford

```
/*
 * Bellman-Ford Algorithm
 * Calculates the shortest distance from a source node, detects negative
     cycles.
 * Complexity: O(V * E)
 */

#include <bits/stdc++.h>
using namespace std;
```

```

const int INF = 1e9;

struct Edge {
    int u, v, w;
};

vector<int> bellman_ford(int n, int src, const vector<Edge>& edges) {
    vector<int> dist(n, INF);
    dist[src] = 0;

    for (int i = 1; i < n; ++i) {
        for (auto [u, v, w] : edges) {
            if (dist[u] < INF && dist[v] > dist[u] + w)
                dist[v] = dist[u] + w;
        }
    }

    // Negative cycle detection
    for (auto [u, v, w] : edges) {
        if (dist[u] < INF && dist[v] > dist[u] + w)
            throw runtime_error("Negative weight cycle detected");
    }

    return dist;
}
```

## 3.4 Conteo de componentes conexas

```
/*
 * Conteo de componentes conexas en grafo no dirigido
 */

#include <bits/stdc++.h>
using namespace std;

int count_components(int n, const vector<vector<int>>& adj) {
    vector<bool> visited(n, false);
    int count = 0;
    for (int u = 0; u < n; ++u) {
        if (!visited[u]) {
            dfs(u, adj, visited);
            ++count;
        }
    }
    return count;
}
```

## 3.5 Union Find

```
/*
 * Union-Find (Disjoint Set Union - DSU)
 * Efficient operations for union and find on disjoint sets.
 * Supports path compression and union by size or rank.
 * Amortized complexity: O(alpha(n)) per operation
 */

#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector<int> parent, size;

    DSU(int n) {
        parent.resize(n);
        size.assign(n, 1);
        iota(parent.begin(), parent.end(), 0); // parent[i] = i
    }

    int find(int u) {
        if (parent[u] != u)
            parent[u] = find(parent[u]); // path compression
        return parent[u];
    }

    bool unite(int u, int v) {
        u = find(u), v = find(v);
        if (u == v) return false;
        if (size[u] < size[v]) swap(u, v); // union by size
        parent[v] = u;
        size[u] += size[v];
        return true;
    }

    bool same(int u, int v) {
        return find(u) == find(v);
```

```
    }
};
```

### 3.6 Graph Diameter

```cpp
// Returns the diameter of an undirected unweighted graph
// n: number of nodes (1-based), edges: list of {u, v}
int graph_diameter(int n, const vector<pair<int, int>>& edges) {
    const int INF = 1e9;
    vector<vector<int>> dist(n + 1, vector<int>(n + 1, INF));
    for (int i = 1; i <= n; i++) dist[i][i] = 0;
    for (auto [u, v] : edges) {
        dist[u][v] = 1;
        dist[v][u] = 1;
    }
    for (int k = 1; k <= n; k++)
        for (int i = 1; i <= n; i++)
            for (int j = 1; j <= n; j++)
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
    int ans = 0;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (dist[i][j] < INF)
                ans = max(ans, dist[i][j]);
    return ans;
}

// Example usage:
// int n = 4;
// vector<pair<int,int>> edges = {{1,2},{2,3},{3,4}};
// int d = graph_diameter(n, edges); // d ==
```

## 4 Manejo de Strings

### 4.1 Sub String

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s = "ProgramacionCompetitiva";

    string sub1 = s.substr(0, 11);  // "Programacion"
    string sub2 = s.substr(11, 11); // "Competitiva"
    string sub3 = s.substr(5);      // from index 5 to the end: "
        amacionCompetitiva"

    cout << sub1 << '\n';
    cout << sub2 << '\n';
    cout << sub3 << '\n';

    return 0;
}
```

### 4.2 KMP

```cpp
/*
 * KMP - Knuth-Morris-Pratt
 * Busca todas las ocurrencias del patron en el texto
 * Complejidad: O(n + m), donde n = tamano del texto, m = tamano del
     patron
 */

vector<int> prefix_function(const string &pat) {
    int m = pat.size();
    vector<int> pi(m);
    for (int i = 1; i < m; ++i) {
        int j = pi[i - 1];
        while (j > 0 && pat[i] != pat[j])
            j = pi[j - 1];
        if (pat[i] == pat[j])
            ++j;
        pi[i] = j;
    }
    return pi;
}

vector<int> kmp_search(const string &text, const string &pat) {
    vector<int> pi = prefix_function(pat);
    vector<int> occ;
    int n = text.size(), m = pat.size(), j = 0;
    for (int i = 0; i < n; ++i) {
        while (j > 0 && text[i] != pat[j])
            j = pi[j - 1];
        if (text[i] == pat[j])
```

```
            ++j;
        if (j == m) {
            occ.push_back(i - m + 1); // ocurrencia encontrada
            j = pi[j - 1];
        }
    }
    return occ;
}

/*
 * Z-Algorithm
 * Z[i] = longitud del prefijo comun entre s y s[i..]
 * Sirve para buscar un patron en texto en O(n + m)
 */

vector<int> z_function(const string &s) {
    int n = s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        if (i <= r)
            z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] - 1 > r)
            l = i, r = i + z[i] - 1;
    }
    return z;
}

// Buscar patron en texto usando Z-function
vector<int> z_search(const string &text, const string &pat) {
    string s = pat + '#' + text;
    vector<int> z = z_function(s);
    vector<int> occ;
    int m = pat.size();
    for (int i = m + 1; i < (int)s.size(); ++i) {
        if (z[i] == m)
            occ.push_back(i - m - 1); // ocurrencia en texto
    }
    return occ;
}
```

# 5 Trucos Varios

## 5.1 Next Permutation

```
//Genera todas las permutaciones en Orden lexicografico
//  La secuencia debe estar previamente ordenada para que funcione
vector<int> v = {1, 2, 3};
do {for (int x : v)cout << x << ' ';
  cout << '\n';
} while (next_permutation(v.begin(), v.end()));
// Salidas 1 2 3 // 1 3 2 // 2 1 3 // 2 3 1 // 3 1 2 // 3 2 1
```

## 5.2 Uso de Fixed

```
#include <iostream>
#include <iomanip> // std::fixed, std::setprecision
using namespace std;

int main() {
    double pi = 3.1415926535;
    cout << fixed << setprecision(3) << pi << '\n'; // Output: 3.142
}
```

## 5.3 Equivante fixed con printf

```
#include <cstdio>

int main() {
    double pi = 3.1415926535;
    printf("%.3f\n", pi); // Output: 3.142
}
```

## 5.4 iota

Para generar arreglos consecutivos

```
#include <numeric>
vector<int> v(5); // [0, 0, 0, 0, 0]
iota(v.begin(), v.end(), 1); // [1, 2, 3, 4, 5]
```

## 5.5 Conversion de unidades

```
/*
 * Conversiones de unidades
```

```
 * Autor: Jairo Flores
 * Conversiones comunes para longitud, masa, tiempo, temperatura, etc.
 */

#include <bits/stdc++.h>
using namespace std;

// Longitud
double km_to_m(double km) { return km * 1000.0; }
double m_to_km(double m) { return m / 1000.0; }
double m_to_cm(double m) { return m * 100.0; }
double cm_to_m(double cm) { return cm / 100.0; }
double inch_to_cm(double in) { return in * 2.54; }
double cm_to_inch(double cm) { return cm / 2.54; }
double ft_to_m(double ft) { return ft * 0.3048; }
double m_to_ft(double m) { return m / 0.3048; }

// Masa
double kg_to_g(double kg) { return kg * 1000.0; }
double g_to_kg(double g) { return g / 1000.0; }
double lb_to_kg(double lb) { return lb * 0.45359237; }
double kg_to_lb(double kg) { return kg / 0.45359237; }

// Tiempo
double hr_to_min(double hr) { return hr * 60.0; }
double min_to_sec(double min) { return min * 60.0; }
double sec_to_ms(double sec) { return sec * 1000.0; }

// Temperatura
double celsius_to_fahrenheit(double c) { return (c * 9.0 / 5.0) + 32.0;
    }
double fahrenheit_to_celsius(double f) { return (f - 32.0) * 5.0 / 9.0;
    }
double celsius_to_kelvin(double c) { return c + 273.15; }
double kelvin_to_celsius(double k) { return k - 273.15; }

// Velocidad
double kmph_to_mps(double kmph) { return kmph / 3.6; }
double mps_to_kmph(double mps) { return mps * 3.6; }
double mph_to_kmph(double mph) { return mph * 1.60934; }
double kmph_to_mph(double kmph) { return kmph / 1.60934; }

// Area
double m2_to_km2(double m2) { return m2 / 1e6; }
double km2_to_m2(double km2) { return km2 * 1e6; }
double acre_to_m2(double acre) { return acre * 4046.85642; }
double m2_to_acre(double m2) { return m2 / 4046.85642; }

// Volumen
double l_to_ml(double l) { return l * 1000.0; }
double ml_to_l(double ml) { return ml / 1000.0; }
double gal_to_l(double gal) { return gal * 3.78541; } // gal (US)
double l_to_gal(double l) { return l / 3.78541; }

// Binario
double kb_to_bytes(double kb) { return kb * 1024.0; }
double mb_to_bytes(double mb) { return mb * 1024.0 * 1024.0; }
double gb_to_bytes(double gb) { return gb * 1024.0 * 1024.0 * 1024.0; }
double bytes_to_kb(double b) { return b / 1024.0; }
double bytes_to_mb(double b) { return b / (1024.0 * 1024.0); }
double bytes_to_gb(double b) { return b / (1024.0 * 1024.0 * 1024.0); }

// Prueba
int main() {
    cout << fixed << setprecision(2);
    cout << "5␣km␣=␣" << km_to_m(5) << "␣m\n";
    cout << "100␣F␣=␣" << fahrenheit_to_celsius(100) << "␣C\n";
    cout << "1␣acre␣=␣" << acre_to_m2(1) << "␣m^2\n";
    cout << "1.5␣GB␣=␣" << gb_to_bytes(1.5) << "␣bytes\n";
    return 0;
}
```

## 5.6 Salidas varias

```
#include <iostream>
using namespace std;

int main() {
    // \n: salto de linea
    cout << "Hola\nMundo" << endl;

    // \t: tabulacion horizontal
    cout << "Nombre\tEdad\nJuan\t20\n";

    // \\: imprime el backslash
    cout << "Ruta:␣C:\\Users\\Jairo\\Documentos\n";
```

```
    // \": imprime comillas dobles
    cout << "Dijo: \"Hola mundo\"\n";

    // \': imprime comillas simples
    cout << "Letra: \'A\'\n";

    // \a: beep (no siempre funciona en todas consolas)
    cout << "\a"; // intento de alerta sonora

    // \b: retroceso (borra el caracter anterior en algunas consolas)
    cout << "ABC\bD\n"; // imprime ABD (borra la C visualmente)

    // \r: retorno de carro (reinicia al inicio de la linea)
    cout << "Sobrescribe\rHola\n"; // Holaescribe

    // \f y \v: no tienen mucho efecto visible en terminales modernas
    cout << "Primera linea\fSegunda (form feed)\n";
    cout << "Primera linea\vSegunda (vertical tab)\n";

    // \0: caracter nulo, marca fin de cadena en arreglos de char
    char palabra[] = {'H', 'o', 'l', 'a', '\0'};
    cout << "Cadena terminada por nulo: " << palabra << endl;

    return 0;
}
```

## 5.7 Aritmetica con Strings

```
#include <bits/stdc++.h>
#include <string>
#include <algorithm>

using namespace std;

// Sum of strings representing positive integers
string sumaStrings(const string &a, const string &b) {
    string res;
    int carry = 0;
    int i = (int)a.size() - 1;
    int j = (int)b.size() - 1;

    while(i >= 0 || j >= 0 || carry) {
        int x = i >= 0 ? a[i] - '0' : 0;
        int y = j >= 0 ? b[j] - '0' : 0;
        int suma = x + y + carry;
        carry = suma / 10;
        res.push_back((suma % 10) + '0');
        i--; j--;
    }
    reverse(res.begin(), res.end());
    return res;
}

// Compare if a >= b (both positive and without leading zeros)
bool mayorIgual(const string &a, const string &b) {
    if (a.size() != b.size()) return a.size() > b.size();
    return a >= b;
}

// Subtract strings a - b (assuming a >= b)
string restaStrings(const string &a, const string &b) {
    if (!mayorIgual(a, b)) return "Error: a < b";

    string res;
    int i = (int)a.size() - 1;
    int j = (int)b.size() - 1;
    int borrow = 0;

    while(i >= 0) {
        int x = a[i] - '0' - borrow;
        int y = j >= 0 ? b[j] - '0' : 0;

        if (x < y) {
            x += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }

        res.push_back((x - y) + '0');
        i--; j--;
    }

    // Remove leading zeros
    while(res.size() > 1 && res.back() == '0') res.pop_back();
```

```
    reverse(res.begin(), res.end());
    return res;
}

// Multiplication of strings
string multiplicacionStrings(const string &a, const string &b) {
    int n = (int)a.size();
    int m = (int)b.size();
    string res(n + m, '0');

    for(int i = n - 1; i >= 0; i--) {
        int carry = 0;
        int x = a[i] - '0';
        for(int j = m - 1; j >= 0; j--) {
            int y = b[j] - '0';
            int suma = (res[i + j + 1] - '0') + x * y + carry;
            res[i + j + 1] = (suma % 10) + '0';
            carry = suma / 10;
        }
        res[i] += carry;
    }

    // Remove leading zeros
    int pos = 0;
    while (pos < (int)res.size() - 1 && res[pos] == '0') pos++;
    return res.substr(pos);
}

// Quick test
int main() {
    string A = "123456789123456789";
    string B = "987654321987654321";

    cout << "Sum: " << sumaStrings(A, B) << "\n";
    cout << "Subtraction (B - A): " << restaStrings(B, A) << "\n";  // B > A
    cout << "Multiplication: " << multiplicacionStrings(A, B) << "\n";

    return 0;
}
```

## 5.8 Aritmetica con Strings segun deepseek

```
#include <iostream>
#include <string>
#include <algorithm>
using namespace std;

// Helper function to remove leading zeros
string removeLeadingZeros(string num) {
    size_t nonZeroIndex = num.find_first_not_of('0');
    if (nonZeroIndex == string::npos) {
        return "0";
    }
    return num.substr(nonZeroIndex);
}

// Addition of two numbers represented as strings
string addStrings(string num1, string num2) {
    num1 = removeLeadingZeros(num1);
    num2 = removeLeadingZeros(num2);

    int i = num1.length() - 1;
    int j = num2.length() - 1;
    int carry = 0;
    string result;

    while (i >= 0 || j >= 0 || carry > 0) {
        int digit1 = (i >= 0) ? num1[i--] - '0' : 0;
        int digit2 = (j >= 0) ? num2[j--] - '0' : 0;
        int sum = digit1 + digit2 + carry;
        carry = sum / 10;
        result.push_back(sum % 10 + '0');
    }

    reverse(result.begin(), result.end());
    return result.empty() ? "0" : result;
}

// Compare two numbers represented as strings (returns 1 if num1 > num2,
    -1 if num1 < num2, 0 if equal)
int compareStrings(string num1, string num2) {
    num1 = removeLeadingZeros(num1);
    num2 = removeLeadingZeros(num2);

    if (num1.length() > num2.length()) return 1;
```

```
    if (num1.length() < num2.length()) return -1;

    for (int i = 0; i < num1.length(); i++) {
        if (num1[i] > num2[i]) return 1;
        if (num1[i] < num2[i]) return -1;
    }

    return 0;
}

// Subtraction of two numbers represented as strings (num1 - num2, assumes num1 >= num2)
string subtractStrings(string num1, string num2) {
    num1 = removeLeadingZeros(num1);
    num2 = removeLeadingZeros(num2);

    int i = num1.length() - 1;
    int j = num2.length() - 1;
    int borrow = 0;
    string result;

    while (i >= 0) {
        int digit1 = num1[i--] - '0' - borrow;
        int digit2 = (j >= 0) ? num2[j--] - '0' : 0;
        borrow = 0;

        if (digit1 < digit2) {
            digit1 += 10;
            borrow = 1;
        }

        result.push_back((digit1 - digit2) + '0');
    }

    reverse(result.begin(), result.end());
    return removeLeadingZeros(result);
}

// Multiplication of two numbers represented as strings
string multiplyStrings(string num1, string num2) {
    num1 = removeLeadingZeros(num1);
    num2 = removeLeadingZeros(num2);

    if (num1 == "0" || num2 == "0") return "0";

    int n1 = num1.length();
    int n2 = num2.length();
    string result(n1 + n2, '0');

    for (int i = n1 - 1; i >= 0; i--) {
        for (int j = n2 - 1; j >= 0; j--) {
            int product = (num1[i] - '0') * (num2[j] - '0') + (result[i + j + 1] - '0');
            result[i + j + 1] = product % 10 + '0';
            result[i + j] += product / 10;
        }
    }

    return removeLeadingZeros(result);
}

// Division of two numbers represented as strings (integer division)
string divideStrings(string num1, string num2) {
    num1 = removeLeadingZeros(num1);
    num2 = removeLeadingZeros(num2);

    if (num2 == "0") return "Error: Division by zero";
    if (compareStrings(num1, num2) < 0) return "0";

    string quotient;
    string current;

    for (int i = 0; i < num1.length(); i++) {
        current += num1[i];
        current = removeLeadingZeros(current);
        if (compareStrings(current, num2) < 0) {
            quotient += "0";
            continue;
        }

        int count = 0;
        string temp = num2;
        while (compareStrings(current, temp) >= 0) {
            temp = addStrings(temp, num2);
            count++;
        }
```

```
        quotient += to_string(count);
        temp = multiplyStrings(num2, to_string(count));
        current = subtractStrings(current, temp);
    }

    quotient = removeLeadingZeros(quotient);
    return quotient.empty() ? "0" : quotient;
}

int main() {
    string num1, num2;
    char op;

    cout << "Enter the first number: ";
    cin >> num1;
    cout << "Enter the second number: ";
    cin >> num2;
    cout << "Enter the operation (+, -, *, /): ";
    cin >> op;

    string result;
    switch(op) {
        case '+':
            result = addStrings(num1, num2);
            break;
        case '-':
            if (compareStrings(num1, num2) < 0) {
                result = "-" + subtractStrings(num2, num1);
            } else {
                result = subtractStrings(num1, num2);
            }
            break;
        case '*':
            result = multiplyStrings(num1, num2);
            break;
        case '/':
            result = divideStrings(num1, num2);
            break;
        default:
            result = "Invalid operation";
    }

    cout << "Result: " << result << endl;
    return 0;
}
```

## 5.9 Problema M codigo

```
#include <bits/stdc++.h>
using namespace std;
vector <int>tabla(10000);
int __find(int x) {
    if (tabla[x] != x)
        tabla[x] = __find(tabla[x]);
    return tabla[x];
}
void __union(int a, int b) {
    a = __find(a);
    b = __find(b);
    if (a != b)
      tabla[b] = a;
}
int main() {
    int n;
    scanf("%d", &n);
    vector<int> dx(n + 10);
    vector<int> dy(n + 10);
    vector<int> dr(n + 10);
    for (int i = 0; i < n;i++) {
        scanf("%d %d %d",&dx[i],&dy[i],&dr[i]);
        tabla[i] = i;
    }
    for (int i = 0; i < n; i++) {
     for (int j = i + 1; j < n; j++) {
            int hx = dx[i] - dx[j];
            int hy = dy[i] - dy[j];
            double tagoras = abs(sqrt(pow(hx,2) + pow(hy,2)));
            if (tagoras <= dr[i] + dr[j]) {
                __union(i, j);
             }
          }
    }
    int ans = 0;
    for (int u = 0;u < n; u++) {
        if (__find(u) == u)
```

```
            ans++;
    }
    printf("%d\n",ans);
}
```

## 5.10 Matriz Viborita

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, m;
    cout << "Enter the number of rows (n): ";
    cin >> n;
    cout << "Enter the number of columns (m): ";
    cin >> m;

    vector<vector<int>> matriz(n, vector<int>(m));
    int valor = 1;

    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) {
            // Left to right
            for (int j = 0; j < m; j++) {
                matriz[i][j] = valor++;
            }
        } else {
            // Right to left
            for (int j = m - 1; j >= 0; j--) {
                matriz[i][j] = valor++;
            }
        }
    }

    // Print the matrix
    cout << "\nSnake matrix:\n";
    for (const auto& fila : matriz) {
        for (int x : fila) {
            cout << x << "\t";
        }
        cout << "\n";
    }

    return 0;
}
```

## 5.11 Triangulo de Pascal

```
#include <iostream>
#include <vector>
#include <iomanip> // for setw
using namespace std;

// Function to calculate combinations C(n, k)
int combinacion(int n, int k) {
    if (k == 0 || k == n) return 1;
    int res = 1;
    for (int i = 1; i <= k; ++i) {
        res *= n--;
        res /= i;
    }
    return res;
}

int main() {
    int filas;
    cout << "Enter the number of rows: ";
    cin >> filas;

    for (int i = 0; i < filas; ++i) {
        // Print spaces to center
        cout << setw((filas - i) * 2);
        for (int j = 0; j <= i; ++j) {
            cout << setw(4) << combinacion(i, j);
        }
        cout << endl;
    }

    return 0;
}
```

# 6 Geometry

## 6.1 Formulas Basicas

```cpp
/*
 * Geometria basica: calculo de areas y subareas
 * Autor: Jairo Flores
 */

#include <bits/stdc++.h>
using namespace std;

// Area de un triangulo dado base y altura
double area_triangulo(double base, double altura) {
    return (base * altura) / 2.0;
}

// Area de un rectangulo dado base y altura
double area_rectangulo(double base, double altura) {
    return base * altura;
}

// Area de un cuadrado dado lado
double area_cuadrado(double lado) {
    return lado * lado;
}

// Area de un circulo dado radio
double area_circulo(double radio) {
    const double PI = acos(-1);
    return PI * radio * radio;
}

// Area de un trapecio (bases mayor y menor, altura)
double area_trapecio(double base_mayor, double base_menor, double altura
    ) {
    return ((base_mayor + base_menor) * altura) / 2.0;
}

// Area de un rombo dado diagonal mayor y menor
double area_rombo(double diagonal_mayor, double diagonal_menor) {
    return (diagonal_mayor * diagonal_menor) / 2.0;
}

// Area de un poligono regular (n lados) con apotema y perimetro
double area_poligono_regular(int n_lados, double apotema, double
    perimetro) {
    return (perimetro * apotema) / 2.0;
}

// Area de un poligono regular usando solo lado y numero de lados (
    formula basada en apotema)
double area_poligono_regular_lado(int n_lados, double lado) {
    const double PI = acos(-1);
    double apotema = lado / (2 * tan(PI / n_lados));
    double perimetro = n_lados * lado;
    return (perimetro * apotema) / 2.0;
}

// Area de sector circular (radio y angulo en grados)
double area_sector_circular(double radio, double angulo_grados) {
    const double PI = acos(-1);
    return (PI * radio * radio) * (angulo_grados / 360.0);
}

// Area de segmento circular (radio y angulo en grados)
double area_segmento_circular(double radio, double angulo_grados) {
    const double PI = acos(-1);
    double angulo_radianes = angulo_grados * PI / 180.0;
    double area_sector = (radio * radio * angulo_radianes) / 2.0;
    double area_triangulo = (radio * radio * sin(angulo_radianes)) /
        2.0;
    return area_sector - area_triangulo;
}

// Perimetro de circulo (circunferencia)
double perimetro_circulo(double radio) {
    const double PI = acos(-1);
    return 2 * PI * radio;
}

// Perimetro de poligono regular (n lados y lado)
double perimetro_poligono_regular(int n_lados, double lado) {
    return n_lados * lado;
}

// Ejemplo de uso
int main() {
    cout << fixed << setprecision(4);
    cout << "Area␣triangulo␣(base=5,␣altura=3):␣" << area_triangulo(5,3)
```

```
        << "\n";
    cout << "Area circulo (radio=2): " << area_circulo(2) << "\n";
    cout << "Area trapecio (bases=3,5, altura=4): " << area_trapecio
        (5,3,4) << "\n";
    cout << "Area poligono regular (hexagono, lado=2): " <<
        area_poligono_regular_lado(6, 2) << "\n";
    cout << "Area sector circular (radio=4, angulo=90): " <<
        area_sector_circular(4, 90) << "\n";
    return 0;
}
```

## 6.2 Convex Hull

```
/*
 * Convex Hull - Algoritmo Graham Scan
 * Autor: Jairo Flores
 * Entrada: vector de puntos (x, y)
 * Salida: vector de puntos que forman el hull convexo en orden ccw
 */

#include <bits/stdc++.h>
using namespace std;

struct Point {
    long long x, y;
    bool operator<(const Point& p) const {
        return y < p.y || (y == p.y && x < p.x);
    }
};

// Producto cruzado (vectorial) de AB x AC
long long cross(const Point &O, const Point &A, const Point &B) {
    return (A.x - O.x)*(B.y - O.y) - (A.y - O.y)*(B.x - O.x);
}

// Funcion que calcula el convex hull y devuelve los vertices en orden
    ccw
vector<Point> convexHull(vector<Point> pts) {
    int n = (int)pts.size();
    if (n <= 1) return pts;

    // Paso 1: encontrar punto base (con menor y, si empate menor x)
    sort(pts.begin(), pts.end());
    Point base = pts[0];

    // Paso 2: ordenar por angulo polar respecto a base
    sort(pts.begin() + 1, pts.end(), [&](const Point &a, const Point &b)
         {
        long long c = cross(base, a, b);
        if (c == 0) // si son colineales, el mas cercano va primero
            return (a.x - base.x)*(a.x - base.x) + (a.y - base.y)*(a.y -
                 base.y) <
                   (b.x - base.x)*(b.x - base.x) + (b.y - base.y)*(b.y -
                        base.y);
        return c > 0;
    });

    // Paso 3: construir el hull usando stack
    vector<Point> hull;
    for (auto &p : pts) {
        while ((int)hull.size() > 1 && cross(hull[hull.size()-2], hull.
            back(), p) <= 0)
            hull.pop_back();
        hull.push_back(p);
    }

    return hull;
}

// Ejemplo de uso
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n; cin >> n;
    vector<Point> pts(n);
    for (int i = 0; i < n; i++)
        cin >> pts[i].x >> pts[i].y;

    vector<Point> hull = convexHull(pts);

    cout << (int)hull.size() << "\n";
    for (auto &p : hull)
        cout << p.x << " " << p.y << "\n";

    return 0;
```

```
}
```

## 6.3 Trigonometria

```
/*
 * Basic trigonometry for competitive programming
 * Author: Jairo Flores
 */

#include <bits/stdc++.h>
using namespace std;

const double PI = acos(-1);

// Trigonometric functions
// Angles in radians (use degrees * PI/180 to convert)

// Convert degrees to radians
double deg_to_rad(double deg) {
    return deg * PI / 180.0;
}

// Convert radians to degrees
double rad_to_deg(double rad) {
    return rad * 180.0 / PI;
}

// Calculate hypotenuse with Pythagoras
double hypotenuse(double a, double b) {
    return sqrt(a*a + b*b);
}

// Law of sines: a/sin(A) = b/sin(B) = c/sin(C)
double law_of_sines(double a, double A_rad, double B_rad) {
    // For example, given side a and angle A, calculates side b given angle B
    return a * sin(B_rad) / sin(A_rad);
}

// Law of cosines: c^2 = a^2 + b^2 - 2ab cos(C)
double law_of_cosines(double a, double b, double C_rad) {
    return sqrt(a*a + b*b - 2*a*b*cos(C_rad));
}

// Area of a triangle with two sides and the angle between them: (1/2)*ab*sin(C)
double triangle_area_trig(double a, double b, double C_rad) {
    return 0.5 * a * b * sin(C_rad);
}

// Angle between vectors (example)
double angle_between_vectors(double x1, double y1, double x2, double y2) {
    // Dot product = |v1||v2|cos(theta)
    double dot = x1*x2 + y1*y2;
    double mag1 = sqrt(x1*x1 + y1*y1);
    double mag2 = sqrt(x2*x2 + y2*y2);
    return acos(dot / (mag1*mag2)); // returns radians
}

// Distance between two points (x1,y1) and (x2,y2)
double distance(double x1, double y1, double x2, double y2) {
    return sqrt((x1 - x2)*(x1 - x2) + (y1 - y2)*(y1 - y2));
}

// Example usage:
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    double A_deg = 30.0, B_deg = 45.0;
    double a = 5.0; // side a
    double A = deg_to_rad(A_deg);
    double B = deg_to_rad(B_deg);

    cout << fixed << setprecision(4);
    cout << "Side b using law of sines: " << law_of_sines(a, A, B) << "\n";

    double c = law_of_cosines(a, law_of_sines(a, A, B), deg_to_rad(60));
    cout << "Side c using law of cosines: " << c << "\n";

    double area = triangle_area_trig(a, law_of_sines(a, A, B), deg_to_rad(60));
    cout << "Triangle area with sides and angle: " << area << "\n";

```

```
    double angle = angle_between_vectors(1, 0, 0, 1);
    cout << "Angle between vectors (1,0) and (0,1): " << rad_to_deg(
        angle) << " degrees\n";

    return 0;
}
```

# 7 Math

## 7.1 Factores de un numero

```
vector<ll> all_factors(ll n) {
vector<ll> fac = {1};
while (n > 1) {
const ll p = mpf[n];
vector<ll> cur = {1};
while (n % p == 0) {
n /= p;
cur.push_back(cur.back() * p);
 }
vector<ll> tmp;
for (auto x : fac)
for (auto y : cur) tmp.push_back(x * y);
  tmp.swap(fac);
}
 return fac;
}
```

## 7.2 Linear Sieve

```
constexpr ll MAXN = 1000000;
bitset<MAXN> is_prime;
 vector<ll> primes;
ll mpf[MAXN], phi[MAXN], mu[MAXN];

void sieve() {
 is_prime.set();
is_prime[1] = 0;
 mu[1] = phi[1] = 1;
for (ll i = 2; i < MAXN; i++) {
 if (is_prime[i]) {
mpf[i] = i;
 primes.push_back(i);
phi[i] = i - 1;
 mu[i] = -1;
}
  for (ll p : primes) {
if (p > mpf[i] || i * p >= MAXN) break;
  is_prime[i * p] = 0;
 mpf[i * p] = p;
  mu[i * p] = -mu[i];
 if (i % p == 0)
  phi[i * p] = phi[i] * p, mu[i * p] = 0;
 else phi[i * p] = phi[i] * (p - 1);
  }
}
```

## 7.3 Cribita normal

```
constexpr int MAXN = 1000000;
bitset<MAXN + 1> is_prime;

void sieve() {
    is_prime.set();      // Mark all as potentially prime
    is_prime[0] = is_prime[1] = 0; // 0 and 1 are not prime

    for (int i = 2; i * i <= MAXN; ++i) {
        if (is_prime[i]) {
            for (int j = i * i; j <= MAXN; j += i)
                is_prime[j] = 0;  // Mark multiples of i as not prime
        }
    }
}
```

## 7.4 Criba vasito

```
int cr[MAXN]; // -1 if prime, some not trivial divisor if not
void init_sieve(){
memset(cr,-1,sizeof(cr));
fore(i,2,MAXN)if(cr[i]<0)for(ll j=1LL*i*i;j<MAXN;j+=i)cr[j]=i;
}
map<int,int> fact(int n){ // must call init_cribe before
map<int,int> r;
while(cr[n]>=0)r[cr[n]]++,n/=cr[n];
if(n>1)r[n]++;
return r;
}
```

## 7.5 Contar Divisores

```cpp
int count_divisors(int n) {
    int res = 1;
    for (int i = 2; i * i <= n; ++i) {
        int count = 0;
        while (n % i == 0) {
            n /= i;
            count++;
        }
        res *= (count + 1);
    }
    if (n > 1) res *= 2; // n es primo > sqrt(original n)
    return res;
}
```

## 7.6 Operaciones con matrices

```cpp
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>

using namespace std;

using Matrix = vector<vector<double>>;

// Function to print matrix
void printMatrix(const Matrix &mat) {
    for (const auto &row : mat) {
        for (double val : row) cout << setw(10) << val << "␣";
        cout << "\n";
    }
}

// Swap rows r1 and r2 in the matrix
void swapRows(Matrix &mat, int r1, int r2) {
    std::swap(mat[r1], mat[r2]);
}

// Calculate determinant using Gaussian elimination
double determinant(Matrix mat) {
    int n = (int)mat.size();
    double det = 1.0;

    for (int i = 0; i < n; i++) {
        // Find row with nonzero pivot
        int pivot = i;
        while (pivot < n && fabs(mat[pivot][i]) < 1e-12) pivot++;
        if (pivot == n) return 0;  // singular matrix

        if (pivot != i) {
            swapRows(mat, i, pivot);
            det = -det;  // change sign if rows are swapped
        }

        det *= mat[i][i];
        if (fabs(mat[i][i]) < 1e-12) return 0;

        // Eliminate below the pivot
        for (int r = i + 1; r < n; r++) {
            double factor = mat[r][i] / mat[i][i];
            for (int c = i; c < n; c++) {
                mat[r][c] -= factor * mat[i][c];
            }
        }
    }

    return det;
}

// Calculate inverse using Gauss-Jordan
bool inverse(const Matrix &input, Matrix &inv) {
    int n = (int)input.size();
    Matrix mat = input;
    inv.assign(n, vector<double>(n, 0));

    // Create identity matrix in inv
    for (int i = 0; i < n; i++) inv[i][i] = 1.0;

    for (int i = 0; i < n; i++) {
        // Find pivot
        int pivot = i;
        for (int r = i + 1; r < n; r++) {
            if (fabs(mat[r][i]) > fabs(mat[pivot][i])) pivot = r;
        }
```

```
        if (fabs(mat[pivot][i]) < 1e-12) return false;  // singular
            matrix

        if (pivot != i) {
            swapRows(mat, i, pivot);
            swapRows(inv, i, pivot);
        }

        double pivot_val = mat[i][i];
        for (int c = 0; c < n; c++) {
            mat[i][c] /= pivot_val;
            inv[i][c] /= pivot_val;
        }

        // Eliminate other elements in column i
        for (int r = 0; r < n; r++) {
            if (r != i) {
                double factor = mat[r][i];
                for (int c = 0; c < n; c++) {
                    mat[r][c] -= factor * mat[i][c];
                    inv[r][c] -= factor * inv[i][c];
                }
            }
        }
    }

    return true;
}

// Example usage
int main() {
    Matrix arr = {
        {4, 7, 2},
        {3, 6, 1},
        {2, 5, 1}
    };

    cout << "Original matrix:\n";
    printMatrix(arr);

    double det = determinant(arr);
    cout << "\nDeterminant: " << det << "\n";

    if (fabs(det) < 1e-12) {
        cout << "Matrix has no inverse (det = 0)\n";
    } else {
        Matrix inv;
        if (inverse(arr, inv)) {
            cout << "\nInverse matrix:\n";
            printMatrix(inv);
        } else {
            cout << "Could not compute the inverse\n";
        }
    }

    return 0;
}
```

## 7.7 Fast Exponentiation

```
def fastExponentiation(base, exponent):
    result = 1
    base = base % MOD
    while exponent > 0:
        if (exponent % 2) == 1:
            result = (result * base) % MOD
        exponent = exponent // 2
        base = (base * base) % MOD
    return result
```

## 7.8 Euler's Totient Function

```
int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0)
                n /= i;
            result -= result / i;
        }
    }
    if (n > 1)
        result -= result / n;
    return result;
}
```

### 7.9 Simpson's Integration

```cpp
const int N = 1000 * 1000; // number of steps (already multiplied by 2)

double simpson_integration(double a, double b){
    double h = (b - a) / N;
    double s = f(a) + f(b); // a = x_0 and b = x_2n
    for (int i = 1; i <= N - 1; ++i) { // Refer to final Simpson's
        formula
        double x = a + h * i;
        s += f(x) * ((i & 1) ? 4 : 2);
    }
    s *= h / 3;
    return s;
}
```

## 8 Mathematical formulas

### 8.1 Sumatorias

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} \tag{1}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} \tag{2}$$

$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2 \tag{3}$$

$$\sum_{i=1}^{n} i^4 = \frac{n^2(n+1)^2(2n+1)}{30} \tag{4}$$